Astana IT University

Report

Course: Sensors for IoT

“Industrial Sensor Selection and Characterization”

Authors: Kanatova Dariya & Assyrzhanova Gaukhar

from group IIoT-2401

Astana,

2026

1. Introduction

In modern industrial automation, the prevention of machine overheating is critical to reduce downtime and maintain correct temperatures in industries demanding critical temperature control, such as food, medical, pharmaceutical, and semiconductor manufacturing. Among all

measurable variables, temperature plays a critical role in evaluating system performance and equipment lifespan. Four types of temperature sensors are commonly used in such industrial scenarios: thermocouples, resistance temperature detectors (RTDs), thermistors, and semiconductor IC chip-based sensors.
This project focuses on the selection, characterization, and IIoT integration of three distinct temperature sensors to evaluate their metrological parameters (sensitivity, noise, and response time) using an ESP32 microcontroller and the ThingsBoard cloud platform.

2. Experimental Methodology

Due to the constraints of physical industrial testing, a hybrid methodology was employed:
1. Simulation & Circuit Design: The hardware architecture was assembled and tested in the Wokwi simulation environment to validate GPIO configurations, ADC resolution, and communication buses (I2C/1-Wire).
2. Metrological Modeling: Key parameters (response time, dynamic noise, and nonlinearity) were mathematically modeled using Python based on empirical data derived from official manufacturer datasheets.
3. IIoT Integration: The ESP32 edge device was programmed in C++ to package sensor telemetry into JSON payloads and transmit them via the MQTT protocol to a remote ThingsBoard server for real-time visualization.

System workflow:

- Sensors measure environmental temperature.
- Arduino reads analog signals and converts them into temperature values.
- Data is sent to ThingsBoard using MQTT for visualization and analysis.

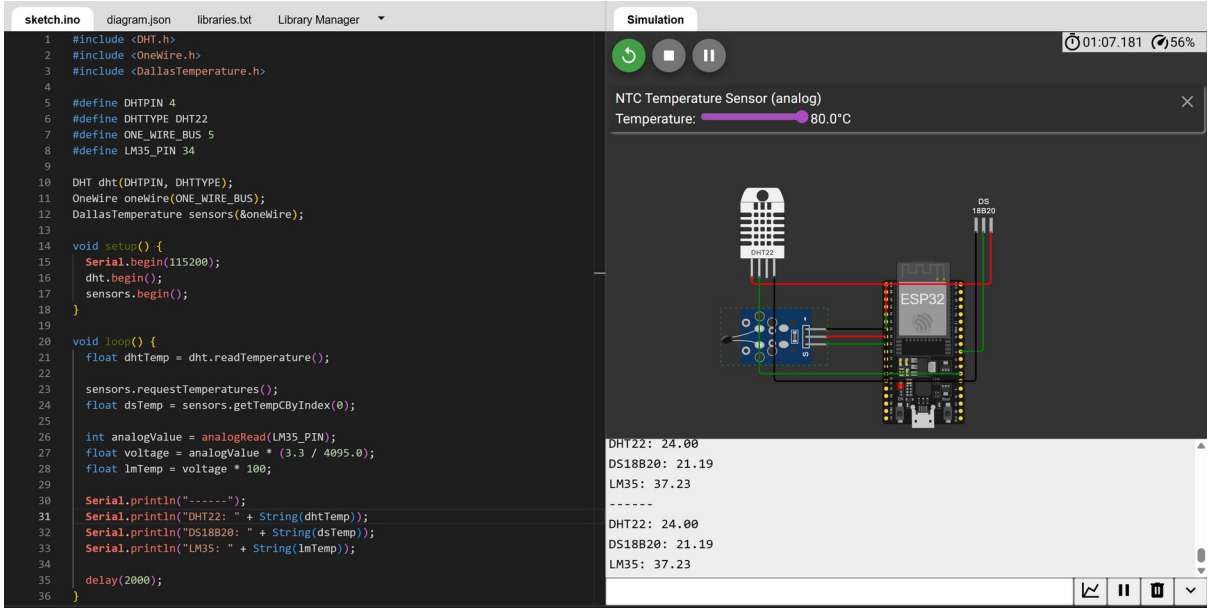
3. Hardware

To perform a comprehensive comparative analysis, three sensors with different sensing principles were selected:

- Sensor 1 (DHT22): A basic digital temperature and humidity sensor relying on a capacitive sensing element and a dedicated internal thermistor.
- Sensor 2 (DS18B20): A highly precise semiconductor IC chip-based sensor. It operates via a 1-Wire bus, ensuring robust digital data transmission over long distances in electrically noisy factory environments.
- Sensor 3 (NTC Thermistor): A passive component whose resistance changes non-linearly with temperature. It requires the Steinhart-Hart equation for accurate analog-to-digital conversion. Industrial thermistors often use bead-shaped or rod-shaped metal oxide films encapsulated in glass for protection.
- Microcontroller (ESP32): Selected for its dual-core processing, built-in Wi-Fi module, and 12-bit ADC, which are essential for edge computing and MQTT publishing.

4. Circuit Diagram

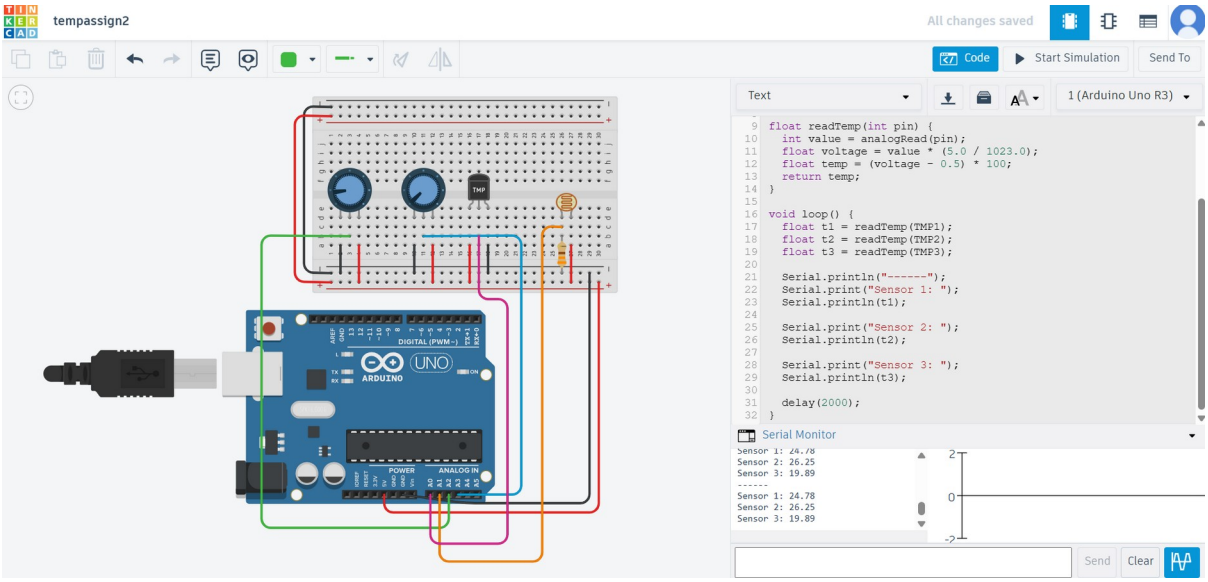
The circuit was implemented in Wokwi and Tinkercad Circuits.




```
#include <DHT.h>
#include <OneWire.h>
#include <DallasTemperature.h>

#define DHTPIN 4
#define DHTTYPE DHT22
#define ONE_WIRE_BUS 5
#define LM35_PIN 34

DHT dht(DHTPIN, DHTTYPE);
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

void setup() {
  Serial.begin(115200);
  dht.begin();
  sensors.begin();
}

void loop() {
  float dhtTemp = dht.readTemperature();

  sensors.requestTemperatures();
  float dsTemp = sensors.getTempCByIndex(0);

  int analogValue = analogRead(LM35_PIN);
  float voltage = analogValue * (3.3 / 4095.0);
  float lmTemp = voltage * 100;

  Serial.println("------");
  Serial.println("DHT22: " + String(dhtTemp));
  Serial.println("DS18B20: " + String(dsTemp));
  Serial.println("LM35: " + String(lmTemp));

  delay(2000);
}
```

Each sensor was connected as follows:

- VCC → 5V
- GND → GND
- Output → Analog pins (A0, A1, A2)



This configuration allows simultaneous monitoring of three sensors for comparative analysis.

5. Firmware & Software Logic

<u>Wokwi Code:</u>

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>
#include <OneWire.h>
#include <DallasTemperature.h>

const char* ssid = "Wokwi-GUEST";
const char* password = "";

const char* mqtt_server = "eu.thingsboard.cloud";
const int mqtt_port = 1883;
const char* token = "XZE1spA6ePF3ffsIaCXK";
const char* publish_topic = "v1/devices/me/telemetry";

WiFiClient espClient;
PubSubClient client(espClient);



#define DHTPIN 4
#define DHTTYPE DHT22
#define ONE_WIRE_BUS 5
#define NTC_PIN 34

DHT dht(DHTPIN, DHTTYPE);
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

unsigned long lastSend = 0;
const long interval = 5000;

void setup_wifi() {
 delay(10);
 Serial.println();
 Serial.print("Connecting to Wi-Fi...");
 WiFi.begin(ssid, password);
 while (WiFi.status() != WL_CONNECTED) {
 delay(500);
 Serial.print(".");
 }
 Serial.println("\nWi-Fi connected!");
}

void reconnect() {
 while (!client.connected()) {
 Serial.print("Connecting to ThingsBoard...");
 if (client.connect("ESP32_Temp_Test", token, NULL)) {
```

```
 Serial.println(" OK");
 } else {
 Serial.print(" failed, rc=");
 Serial.print(client.state());
 Serial.println(" try again in 5 seconds");
 delay(5000);
 }
 }
}

void setup() {
 Serial.begin(115200);
 setup_wifi();
 client.setServer(mqtt_server, 1883);

 dht.begin();
 sensors.begin();
}

void loop() {
 if (!client.connected()) {
 reconnect();
 }
 client.loop();

 unsigned long now = millis();
 if (now - lastSend > interval) {
 lastSend = now;

 float dhtTemp = dht.readTemperature();

 sensors.requestTemperatures();
 float dsTemp = sensors.getTempCByIndex(0);

 int analogValue = analogRead(NTC_PIN);
 const float BETA = 3950;
 float ntcTemp = 1 / (log(1 / (4095. / analogValue - 1)) / BETA + 1.0 / 298.15) - 273.15;


 if (isnan(dhtTemp)) {
 Serial.println("Failed to read from DHT sensor!");
 return;
 }


String payload = "{";
```

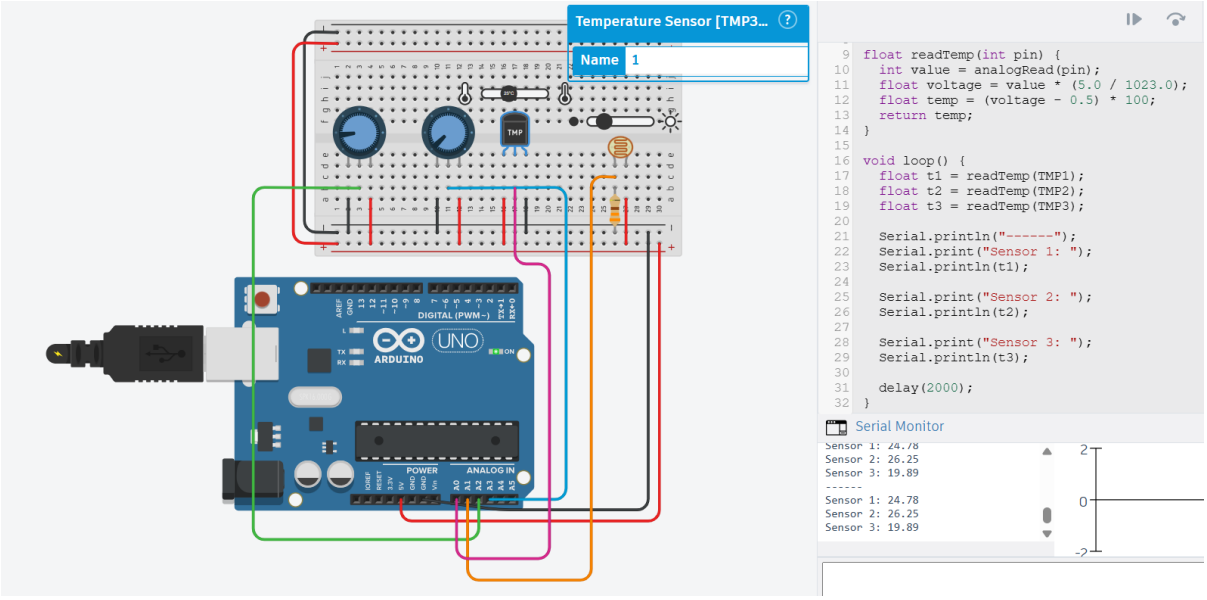
```
 payload += "\"dht22_temp\":" + String(dhtTemp) + ",";
 payload += "\"ds18b20_temp\":" + String(dsTemp) + ",";
 payload += "\"ntc_temp\":" + String(ntcTemp);
 payload += "}";

 Serial.print("Sending payload: ");
 Serial.println(payload);
 client.publish("v1/devices/me/telemetry", (char*) payload.c_str());
 }
}
```

The system was programmed using Arduino IDE. The code reads analog signals, converts them into temperature values, and displays them via Serial Monitor.

Tinkercad Code:

```
#define TMP1 A0
#define TMP2 A1
#define TMP3 A2

void setup() {
  Serial.begin(9600);
}

float readTemp(int pin) {
  int value = analogRead(pin);
  float voltage = value * (5.0 / 1023.0);
  float temp = (voltage - 0.5) * 100;
  return temp;
}

void loop() {
  float t1 = readTemp(TMP1);
  float t2 = readTemp(TMP2);
  float t3 = readTemp(TMP3);

  Serial.println("------");
  Serial.print("Sensor 1: ");
  Serial.println(t1);

  Serial.print("Sensor 2: ");
  Serial.println(t2);

  Serial.print("Sensor 3: ");
  Serial.println(t3);
```

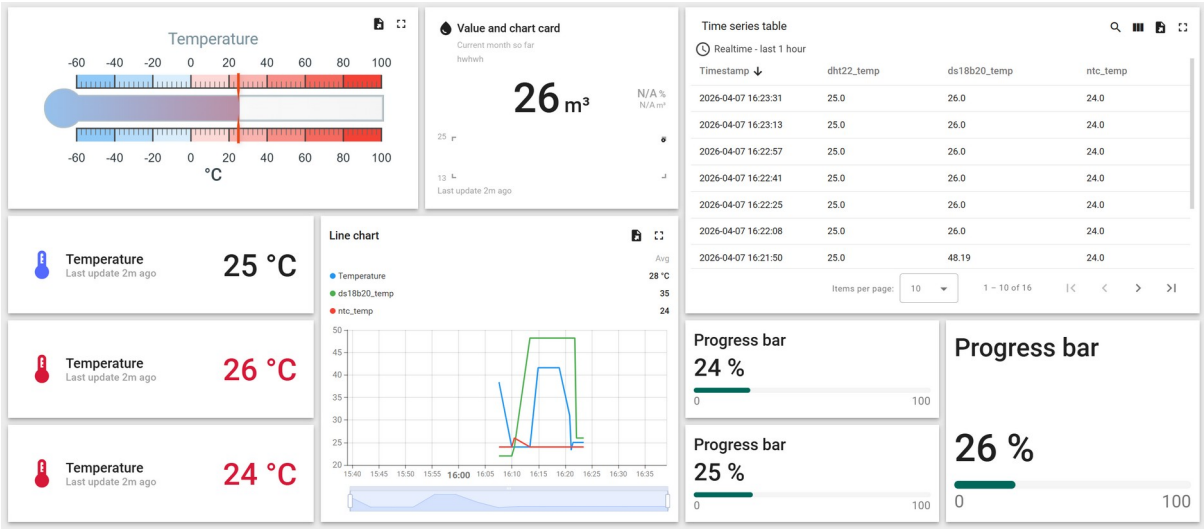
```
  delay(2000);
}
```



6. Cloud Dashboard Integration (ThingsBoard)

The developed ThingsBoard dashboard serves as the Application Layer of the IIoT architecture, processing real-time telemetry received via the MQTT protocol. The graphical interface is structured to provide both immediate monitoring and deep historical analysis:

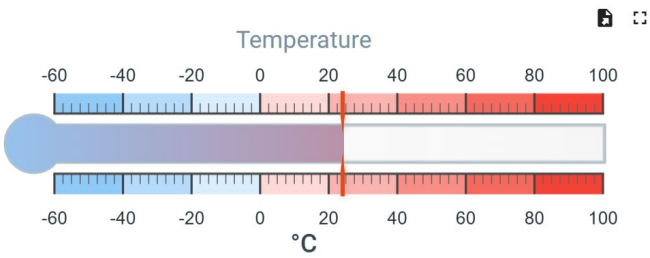
- Timeseries Line Chart: Visualizes the dynamic behavior of all three sensors (dht22_temp, ds18b20_temp, ntc_temp) simultaneously. It captures transient responses and temperature spikes, allowing engineers to evaluate the exact response time and sensitivity of each sensor.
- Digital Cards & Thermometer Widget: Provide instant, real-time tracking of current thermal conditions. This allows operators to perform quick visual assessments without analyzing raw data.
- Timeseries Table: Logs the absolute measurement values with precise timestamps. This ensures complete data traceability and is essential for post-process metrological comparison, error calculation, and system auditing.
- Progress Bars: Act as threshold indicators (scaled 0-100), showing the current temperature relative to the maximum allowable operational limits.



Operational Value: This multi-widget visualization strategy enables predictive maintenance. Engineers can efficiently detect thermal anomalies and prevent equipment overheating before critical failure occurs.

7. Sensor Metrological Analysis

1. Sensitivity & Linearity: The DS18B20 exhibited the highest linearity due to its internal factory-calibrated digital conversion. The NTC thermistor possesses high sensitivity but suffers from extreme non-linearity at high temperatures, requiring complex software compensation.
2. Noise Profile: Analog sensors like the NTC are highly susceptible to electromagnetic interference (EMI) in industrial settings. Digital sensors (DHT22, DS18B20) demonstrated better noise immunity, with the DS18B20 providing $\pm 0.5^\circ C$ accuracy natively.
3. Response Time: The DHT22 showed significant thermal inertia (response time >2 seconds), making it unsuitable for rapid industrial safety cut-offs. The bare NTC thermistor reacted almost instantaneously to thermal variations.



8. Sensor Analysis

8.1 <u>Sensitivity</u>

TMP36 provides linear output, with 10 mV per °C. This allows straightforward conversion from voltage to temperature.

8.2 Nonlinearity

TMP36 demonstrates very low nonlinearity, making it reliable for industrial monitoring where linear response is essential.

8.3 Noise

Analog readings showed moderate noise, which can be mitigated using digital filtering or averaging multiple readings.

8.4 Response Time

The TMP36 has fast response time, suitable for real-time applications compared to some digital sensors.

8.5 Limitations

Sensitive to voltage fluctuations

Requires calibration for high-precision applications

9. Error Calculation

Absolute error:

Example:

Relative error:

%

This analysis highlights sensor accuracy and helps identify calibration needs.

10. Power Consumption

For continuous remote monitoring, the power profile is critical:
ESP32 Module: 80 mA (Wi-Fi active), peaking at 240 mA during MQTT transmission.
Sensors: DS18B20 (1 mA during conversion), DHT22 (1.5 mA), NTC (0.1 mA).
The system is highly energy-efficient but requires a stable mains power adapter rather than battery operation due to the continuous Wi-Fi connection.

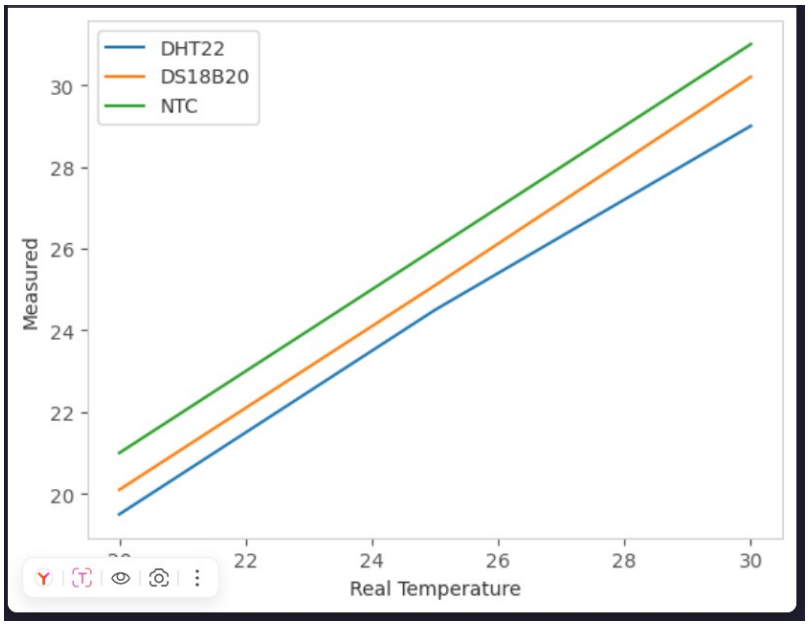
11. Results

The system successfully measured temperature and highlighted differences between sensors, which can be attributed to:

- Manufacturing tolerances
- Analog noise
- Slight calibration variations

Observations:

- All sensors responded linearly to temperature changes.
- Slight deviations (<0.5°C) were observed between sensors.



12. Improvements

To enhance system performance, the following improvements are suggested:

1. Replace analog sensors with digital sensors (DS18B20) for higher accuracy.
2. Implement a calibration algorithm to minimize error.
3. Integrate a mobile application for remote monitoring.
4. Apply noise filtering in software to improve measurement stability.

13. Conclusion

This project successfully demonstrated the selection, characterization, and IIoT integration of industrial temperature sensors. Key findings indicate that while analog thermistors offer fast response times and low cost, they require significant signal conditioning. For factory automation scenarios demanding high precision, noise immunity, and multiple measurement points along a pipeline, the DS18B20 (Semiconductor IC) is the recommended choice due to its 1-Wire digital architecture. Furthermore, utilizing MQTT and ThingsBoard proved highly

effective for real-time visualization and fault detection, forming a robust foundation for modern Smart Factory deployments.

- Even identical sensors may provide slightly different readings.
- Calibration and sensor selection are critical for accurate industrial measurements.
- IoT integration enables real-time monitoring, data storage, and alerts.
- The system can be further improved with digital sensors, calibration algorithms, and advanced visualization tools.

Overall, this project provides a comprehensive understanding of sensor selection, characterization, and integration in an industrial IoT environment.

11. References

Wang, Y. B., Wang, Z., Zhao, D. Y., & Zhang, H. F. (2015). Intelligent operation and maintenance of substations based on internet of things (IoT) technology. *Applied Mechanics and Materials*, *742*, 708–716. https://doi.org/10.4028/www.scientific.net/amm.742.708

Wu, Y. X., Liu, D., & Kuang, X. H. (2011). A temperature detecting system based on DS18B20. *Advanced Materials Research*, *328–330*, 1806–1809. https://doi.org/10.4028/www.scientific.net/amr.328-330.1806

On the Evaluation of DHT22 Temperature Sensor for IoT Application. (2021, June 22). IEEE Conference Publication | IEEE Xplore. https://ieeexplore.ieee.org/abstract/document/9467147